



VIVI · ViviendAI

Documentación técnica integral, auditoría, arquitectura, modelo de datos, agentes de IA y hoja de ruta

PERFILAMIENTO INTELIGENTE DE LEADS DE VIVIENDA

Versión auditada: 26 de julio de 2026

Estado: prototipo funcional con integraciones simuladas

Validación: código, datos, SQLite, Make, Gemini, Telegram, NotebookLM y pruebas automatizadas

Documento de ingeniería y producto. El score es una prioridad comercial explicable; no equivale a aprobación de crédito ni asignación de subsidio.

Tablero ejecutivo de la auditoría

Pruebas automatizadas

20/20

Compilación y suite unitarias correctas

Registros históricos

4.142

26 proyectos analizados

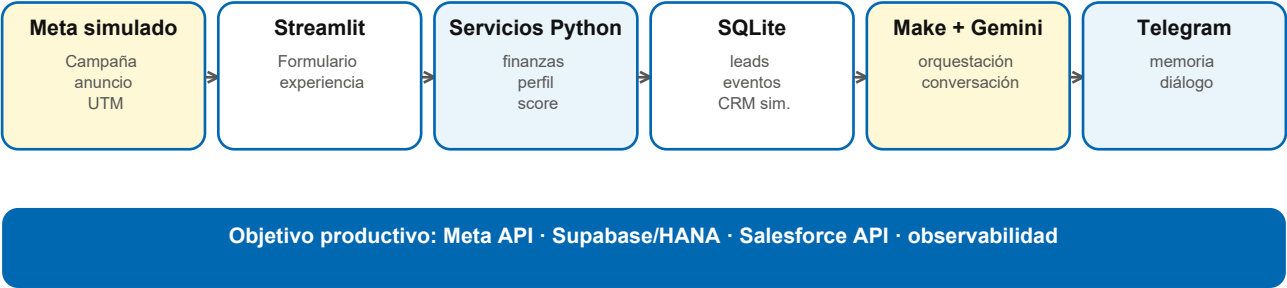
Score explicable

0–100

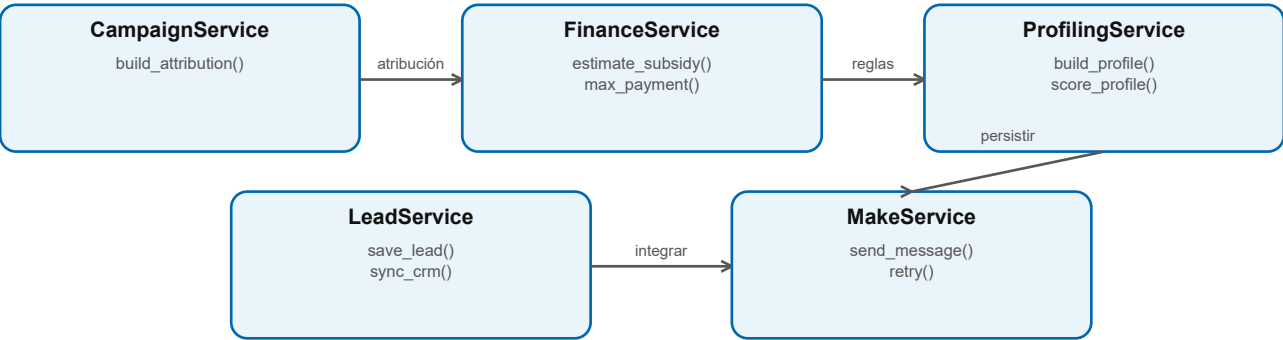
Seis dimensiones determinísticas

Capacidad	Estado	Lectura ejecutiva
Captura y atribución	Implementada	Proyecto, campaña, anuncio y UTM
Finanzas y scoring	Implementada	Reglas auditable; sin decisión crediticia
Telegram + memoria	Parcial	Falta extracción estructurada y rescore
CRM/HANA/Supabase	Simulado/objetivo	No existe conexión productiva
Separación	Pendiente	Debe incorporarse a la demo

Arquitectura lógica verificada



UML de componentes del backend



La IA conversa y extrae señales; las reglas financieras y el score deben permanecer fuera del modelo generativo. Esta separación reduce alucinaciones y permite auditar cada punto.

Fecha de corte: 26 de julio de 2026

Naturaleza: prototipo demostrativo del reto de vivienda de Colsubsidio

Resultado general: funcional con brechas controladas; no es una integración productiva.

1. Resumen ejecutivo

VIVI convierte un registro originado en una campaña específica de Meta en un lead enriquecido con trazabilidad de campaña, estimaciones financieras, una conversación empática, una Ficha del Sueño y un score comercial explicable. Streamlit representa la experiencia; SQLite emula la persistencia empresarial; Make coordina Gemini y Telegram; Salesforce se representa mediante un payload local.

La auditoría verificó el código, los datos, la base SQLite, los blueprints de Make, los prompts de Gemini, la configuración, el modelo de scoring y las pruebas. También contrastó el resultado con el cuaderno NotebookLM del proyecto y con documentación oficial de Gemini.

El prototipo cumple bien la captura contextual, la estimación determinística, el score auditable y la experiencia de demostración. Las principales brechas son:

1. La conversación de Telegram todavía no actualiza el perfil estructurado ni vuelve a calcular el score determinístico.
2. Supabase, SAP HANA Cloud, Salesforce, Meta e Instagram son objetivos o simulaciones; no integraciones productivas.
3. El flujo demostrado termina en el CRM simulado; falta representar la cita, viabilidad financiera y separación inferior a \$1.000.000.
4. El inicio de Telegram usa un código de lead, no un token firmado y expirable.
5. El escenario Streamlit consume una llamada Gemini al inicializar memoria, lo que agrava el límite gratuito de solicitudes.

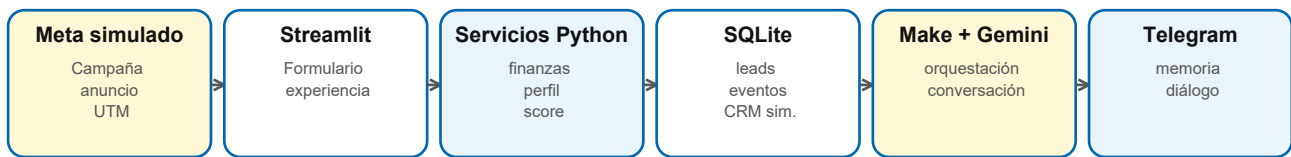
2. Fuentes de verdad verificadas

- Cuaderno NotebookLM LeadFlow 30X – Captura y calificación financiera.
- Fuente del reto RETO DE VIVIENDA.
- Reunión del reto y notas de Gemini del 24 de julio de 2026.
- Página de subsidio de vivienda Colsubsidio para 2026.
- Código y datos locales de Prototipo.
- Blueprints exportados de Make.
- Documentación oficial de Gemini sobre modelos, límites, errores y salida estructurada.

NotebookLM confirmó como reglas centrales:

- conservar campaña, anuncio, proyecto y UTM;
- priorizar afiliados por la regla 90/10 sin excluir a no afiliados;
- máximo de cuatro SMMLV para subsidio;
- 30 SMMLV para ingresos de hasta dos SMMLV y 20 SMMLV entre dos y cuatro;
- cuota hipotecaria orientativa máxima del 40 % del ingreso del hogar;
- conversación humana, breve y de una pregunta por turno;
- score de 0 a 100 determinístico y explicable;
- filtrar y enriquecer antes de Salesforce;
- alcance del reto hasta la separación del inmueble.

3. Inventario técnico



Objetivo productivo: Meta API · Supabase/HANA · Salesforce API · observabilidad

Aplicación

- `app.py`: interfaz Streamlit y orquestación de la demo.
- `campaign_service.py`: atribución de campaña, anuncio, formulario y UTM.
- `catalog_service.py`: catálogo y perfiles históricos por proyecto.
- `finance_service.py`: subsidio y capacidad de pago.
- `instagram_simulator.py`: conversación local y extracción básica.
- `lead_service.py`: SQLite, eventos, deduplicación y Salesforce simulado.
- `make_service.py`: cliente HTTP para Make con timeout, reintentos y API key opcional.
- `profiling_service.py`: perfil estructurado y scoring VIVI-1.0.

Datos

- `tableConvert.com_x950qq.json`: 4.142 registros históricos y 26 proyectos.
- `perfiles_compradores_por_proyecto.json`: perfiles agregados por proyecto.
- `proyectos.json`: catálogo normalizado de proyectos.
- `leads.db`: base local del prototipo.

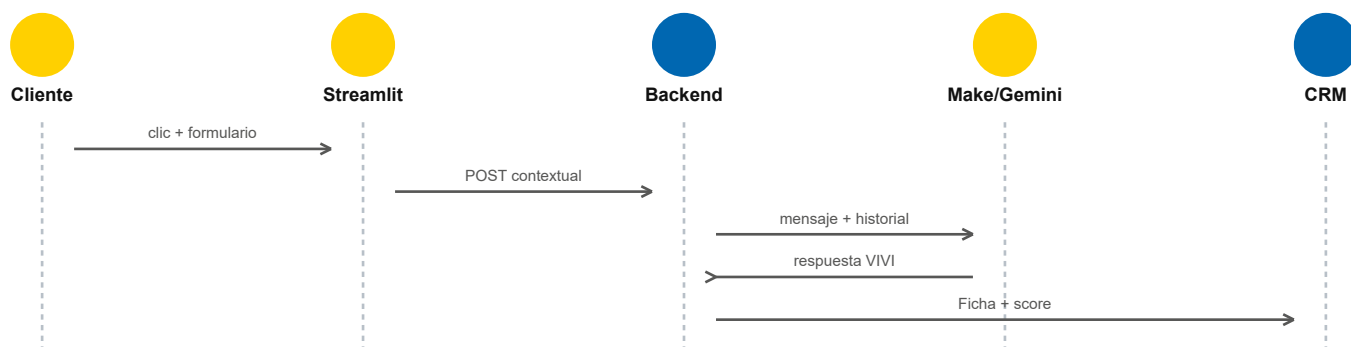
Configuración

- `config/agents.json`: roles y restricciones de los agentes.
- `config/scoring.json`: dimensiones, umbrales y variables prohibidas.
- `config/salesforce_fields.json`: contrato de salida al CRM.
- `schemas/lead_profile.schema.json`: contrato JSON del perfil.
- `.env`: secretos locales; nunca debe versionarse.

Automatización

- `Integration Telegram Bot.blueprint.json`.
- `Integration Streamlit Gemini.blueprint.json`.
- Make Data Store VivierendAI_DATA como memoria temporal.

4. Flujo implementado



1. El usuario selecciona un proyecto y canal en Streamlit.
2. La campaña genera identificadores de campaña, anuncio, formulario y UTM.
3. El formulario captura identificación, ingresos, afiliación, horizonte, ahorro, habitaciones y consentimiento.
4. El backend valida, normaliza, asigna LEAD-xxxxx y registra el lead en SQLite.
5. Las reglas financieras calculan subsidio potencial y cuota orientativa.
6. El scoring inicial se calcula de forma determinística.
7. Make/Gemini inicia o continúa la conversación en Instagram simulado o Telegram.
8. La interfaz presenta propensión, prioridad, ruta y Ficha del Sueño.
9. Cuando corresponde, se construye un payload de Salesforce simulado.

5. Modelo de scoring VIVI-1.0

Dimensión	Máximo
Intención de compra	25
Horizonte	15
Preparación financiera	25
Encaje con proyecto	15
Participación conversacional	10
Aceptación del siguiente paso	10
Total	100

Puntaje	Prioridad	Nota
80–100	ALTA	Asesor comercial
55–79	MEDIA	Completar perfil
30–54	NUTRICIÓN	Pertenecer
0–29	TEMPRANO	Nutrición digital

El resultado contiene desglose, razones, datos faltantes y acción recomendada. No usa género, etnia, discapacidad, dirección exacta, empleador o presencia en redes para puntuar. El score orienta la prioridad; no aprueba créditos ni subsidios.

6. Reglas financieras

Con SMMLV 2026 de \$1.750.905:

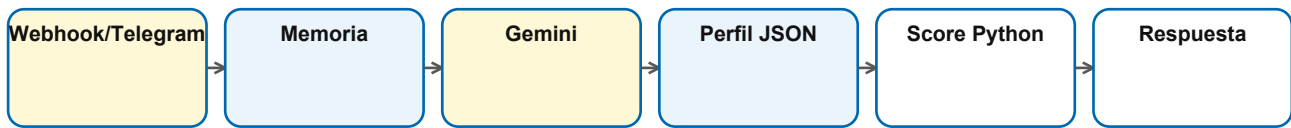
- ingresos mayores que cero y hasta 2 SMMLV: subsidio potencial de 30 SMMLV;
- más de 2 y hasta 4 SMMLV: subsidio potencial de 20 SMMLV;
- más de 4 SMMLV: sin subsidio familiar bajo esta regla;
- cuota máxima orientativa: 40 % del ingreso mensual del hogar;
- subsidio concurrente se presenta como potencial y sujeto a validación

externa;

- no se promete asignación y no se usa crédito de libre inversión para subsidio.

Para una decisión real faltan validaciones de afiliación activa, categoría, antigüedad mínima, aportes, propiedad de vivienda, subsidios previos, hogar y viabilidad hipotecaria.

7. Auditoría de Make y Gemini



Actual: conversación y memoria; el perfil/score posterior aún no se cierra en Telegram.

Objetivo: salida estructurada validada + scoring determinístico + persistencia permanente.

Escenario Telegram

Flujo actual:

Telegram Watch Updates → Data Store Get Record → Gemini → Telegram Reply → Data Store Add/Replace Record

Fortalezas:

- ejecución inmediata;
- procesamiento secuencial para evitar conversaciones cruzadas;
- memoria por chat y recuperación especial por código LEAD-;
- modelo Flash Lite, respuesta corta y una pregunta por turno;
- no solicita datos bancarios, dirección exacta ni centrales de riesgo.

Brechas:

- no filtra actualizaciones sin mensaje de texto;
- no tiene manejador específico para 429/5xx;
- no extrae un JSON estructurado después de cada turno;
- no llama al motor determinístico para recalcular el score;
- no persiste el perfil final en Supabase ni SQLite;
- cambia la clave de memoria de lead a chat sin una tabla explícita de enlace;
- el código /start no está firmado ni expira.

Escenario Streamlit/Instagram simulado

Flujo actual:

Custom webhook → Gemini → Data Store Add/Replace → Webhook response

Fortalezas:

- ejecución inmediata compatible con Webhook Response;
- respuesta síncrona para la interfaz;
- memoria por lead_id;
- contexto de campaña y proyecto.

Brechas:

- no consulta la memoria en Make; Streamlit debe enviar el historial;

- la inicialización consume Gemini aunque solo se necesita guardar memoria;
- no hay autenticación obligatoria del webhook en el blueprint;
- no hay ruta de fallback cuando Gemini devuelve 429;
- la respuesta es texto, no un contrato JSON versionado.

Recomendación de arquitectura Make

Separar tres operaciones:

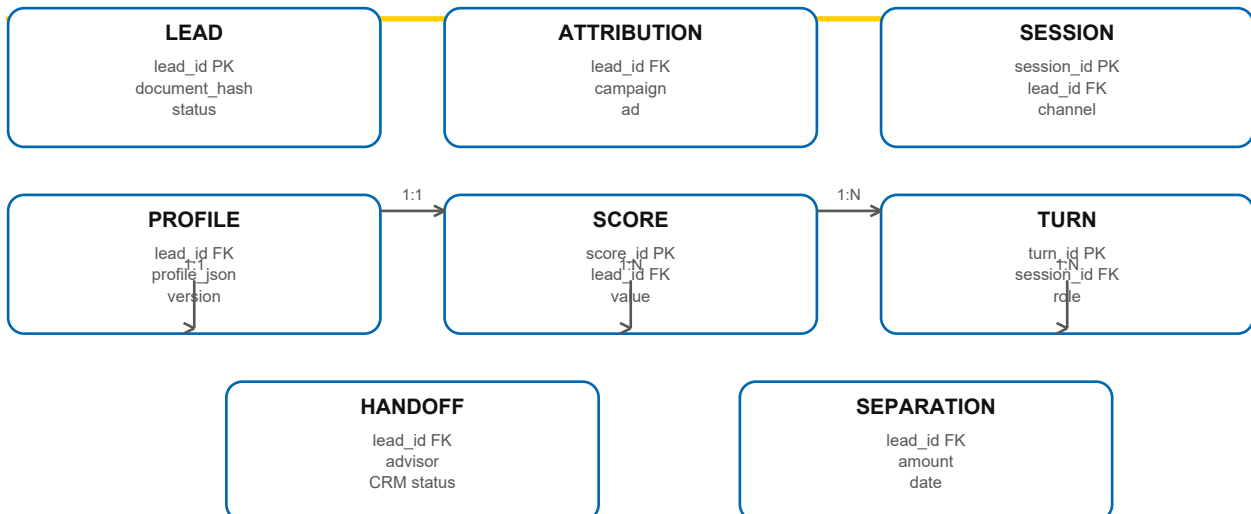
1. Inicializar memoria: webhook sin Gemini que registra lead, proyecto, campaña y perfil inicial.
2. Conversar: recuperar memoria, llamar Gemini y guardar el turno.
3. Perfilar: salida estructurada de Gemini, validación JSON, scoring determinístico y persistencia permanente.

El webhook que responde a Streamlit debe ejecutarse `Immediately`. En ese escenario `Process data in order` debe permanecer desactivado porque Make no admite Webhook Response con procesamiento secuencial. Telegram sí debe conservar procesamiento secuencial.

Gestión de cuotas Gemini

- usar un modelo Flash Lite disponible y estable;
- eliminar parámetros de muestreo obsoletos en modelos 3.5;
- limitar el historial a un resumen más los últimos turnos;
- máximo de salida reducido;
- evitar llamadas Gemini para tareas determinísticas o de almacenamiento;
- reintento exponencial con `Retry-After` para 429 y errores transitorios;
- mensaje de fallback y cola de reintento, sin dejar al cliente sin respuesta;
- monitorear solicitudes por minuto y por día por proyecto/modelo.

8. Persistencia y modelo de datos



SQLite implementado

META_LEADS_CAPTURE almacena identidad, atribución, datos financieros declarados, perfil, score, estado de embudo, CRM y marcas de tiempo.

INTEGRATION_EVENTS conserva la trazabilidad técnica del flujo sin duplicar PII completa en el payload de eventos.

Make Data Store implementado

Memoria temporal:

- key;
- lead_id;
- project_origin;
- history;
- turn_count;
- profile_json;
- last_score;
- updated_at.

Supabase objetivo

Debe convertirse en fuente permanente multicanal con tablas:

- leads;
- lead_attribution;
- conversation_sessions;
- conversation_turns;
- lead_profiles;
- score_snapshots;
- integration_events;
- advisor_handoffs;
- appointments;
- separations.

9. Calidad, seguridad y privacidad

Correcciones realizadas durante la auditoría:

- ingreso cero ya no genera subsidio;
- la extracción de intención evita coincidencias parciales como si en sin;
- la deduplicación usa documento cuando está disponible;
- los eventos ya no replican todo el formulario con PII;
- el reintento a CRM no omite el requisito de perfil completado;
- el mapeo de Salesforce quedó alineado con campaña, consentimiento y score;
- el esquema JSON fue alineado con el perfil real;
- proyectos.json quedó como JSON válido;
- el cliente Make reintenta 429 y errores transitorios;
- el blueprint Telegram usa Flash Lite y menos tokens.

Pendientes antes de exposición pública:

- rotar todas las claves mostradas durante el desarrollo;
- reemplazar el secreto placeholder de Telegram;
- activar autenticación del webhook de Make;
- cifrar o tokenizar documentos;
- aplicar retención y borrado verificable;
- registrar consentimiento con versión de política;
- controlar acceso por rol y auditar consultas;
- no utilizar Sherlock/Holehe para scoring: implican enriquecimiento externo

invasivo, sesgo, exactitud incierta y riesgos de tratamiento de datos.

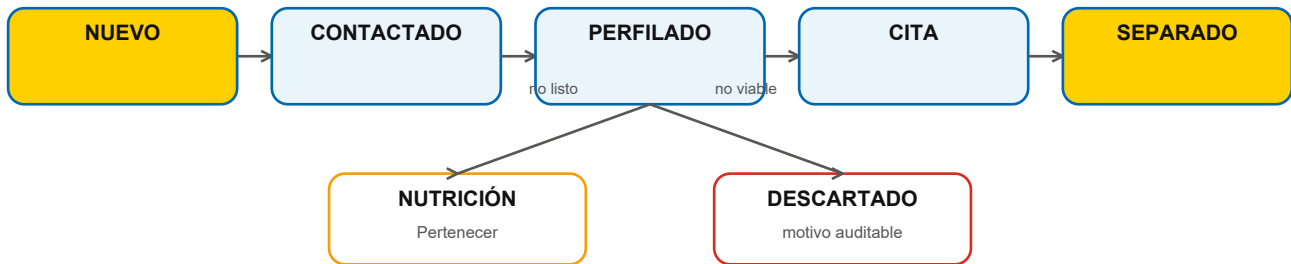
10. Verificación ejecutada

- compilación de módulos Python: correcta;
- 20 pruebas unitarias: correctas;
- seis archivos JSON: válidos;
- perfil vacío de ejecución: válido contra JSON Schema;
- AppTest de Streamlit: sin excepciones;
- rutas locales: retiradas de la experiencia visible;
- base SQLite: migraciones y eventos funcionales.

11. Matriz de cumplimiento

Capacidad	Estado	Observación
Atribución Meta/UTM	Implementada	Simulada y trazable
Formulario contextual	Implementada	No repite proyecto
SQLite como gemelo HANA	Implementada	Solo local
Finanzas determinísticas	Implementada	Orientativa
Score auditable	Implementada	Recalcular tras Telegram
Instagram simulado	Implementada	Síncrono vía Make
Telegram con memoria	Parcial	Falta perfil estructurado
Ficha del Sueño	Implementada	Evoluciona en interfaz
Salesforce	Simulada	Payload local
Supabase	Diseñada	No integrada
Afiliación real	Diseñada	Sin cruce productivo
Separación	Diseñada	Falta etapa demostrable
RAG comprador/proyectos	Parcial	Datos cargados, sin agente RAG
Voz/Gemini Live	Hoja de ruta	No necesaria para MVP

12. Hoja de ruta priorizada



P0 — antes de la demo

1. Rotar secretos y proteger webhooks.
2. Añadir fallback de 429 en ambos escenarios.
3. Crear rama de inicialización sin Gemini.
4. Añadir filtro de mensajes válidos en Telegram.
5. Probar un recorrido limpio con un lead nuevo y documentar evidencias.

P1 — cerrar el perfilamiento

1. Crear Agente 2 con salida JSON estructurada.
2. Validar el JSON contra el esquema.
3. Ejecutar el score Python como única fuente de puntuación.
4. Persistir perfil y score en una base permanente.
5. Enviar a Salesforce solo al cumplir la regla de enrutamiento.

P2 — cubrir el alcance completo

1. Simular contacto de asesor y agendamiento.
2. Registrar viabilidad hipotecaria como estado, no como promesa.
3. Incorporar inventario recomendado.
4. Simular separación inferior a \$1.000.000.
5. Medir conversión por campaña hasta separación.

P3 — producción

1. Supabase/HANA con cifrado, RLS y retención.
2. OAuth y APIs reales de Meta/Salesforce.
3. observabilidad, colas, idempotencia y recuperación;
4. evaluación del agente, pruebas de carga y control de costos;
5. gobierno de modelos, prompts y versiones de scoring.

13. Criterio de aceptación de la demo

La demostración está lista cuando un lead nuevo:

1. conserva proyecto/campaña/UTM;
2. completa el formulario y consentimiento;
3. recibe estimaciones sin promesas;
4. conversa con VIVI sin repetir datos;
5. produce perfil JSON válido;
6. recibe score determinístico con razones;
7. se enruta según prioridad;
8. genera Ficha del Sueño;
9. llega al CRM simulado con trazabilidad;
10. puede avanzar en una simulación hasta separación.

14. Referencias externas

- Google AI, límites de Gemini:

<https://ai.google.dev/gemini-api/docs/rate-limits>

- Google AI, modelos actuales:

<https://ai.google.dev/gemini-api/docs/latest-model>

- Google AI, salida estructurada:

<https://ai.google.dev/gemini-api/docs/structured-output>

- Google AI, solución de errores:

<https://ai.google.dev/gemini-api/docs/troubleshooting>

- Innovación Colsubsidio:

<https://innovacion.colsubsidio.com/>